

Operating systems

Cooperating Process

Content Lecture#9

01 Concurrent process

02 Cooperating Process

03 Critical Section

04 Solution to critical Section

05 Invalid Solution to critical section

Lets take a Break

A programmer had a problem, so he used threads. Then he had two problems.







Cooperating Process

A process is cooperating if it can affect or be affected by the other processes executing in the system

Critical Section

Part of the program where shared memory is accessed is called critical section

Concurrent Process

- Many processes can be executed concurrently to perform multitasking on a CPU due to following reasons
  - To share system resources in a multi-user environment
  - To share data/database
  - To breakup a task into subtasks, each execute in parallel
  - Parallel execution of tasks like editing, spell checking, printing and compiling
- In concurrency, the issues of cooperation among the processes, synchronization and communication are very important
- Such a process has the following characteristics
  - Its state is shared among other processes
  - The result of its execution cannot be predicted because of relative execution
  - The result of its execution is non deterministic.

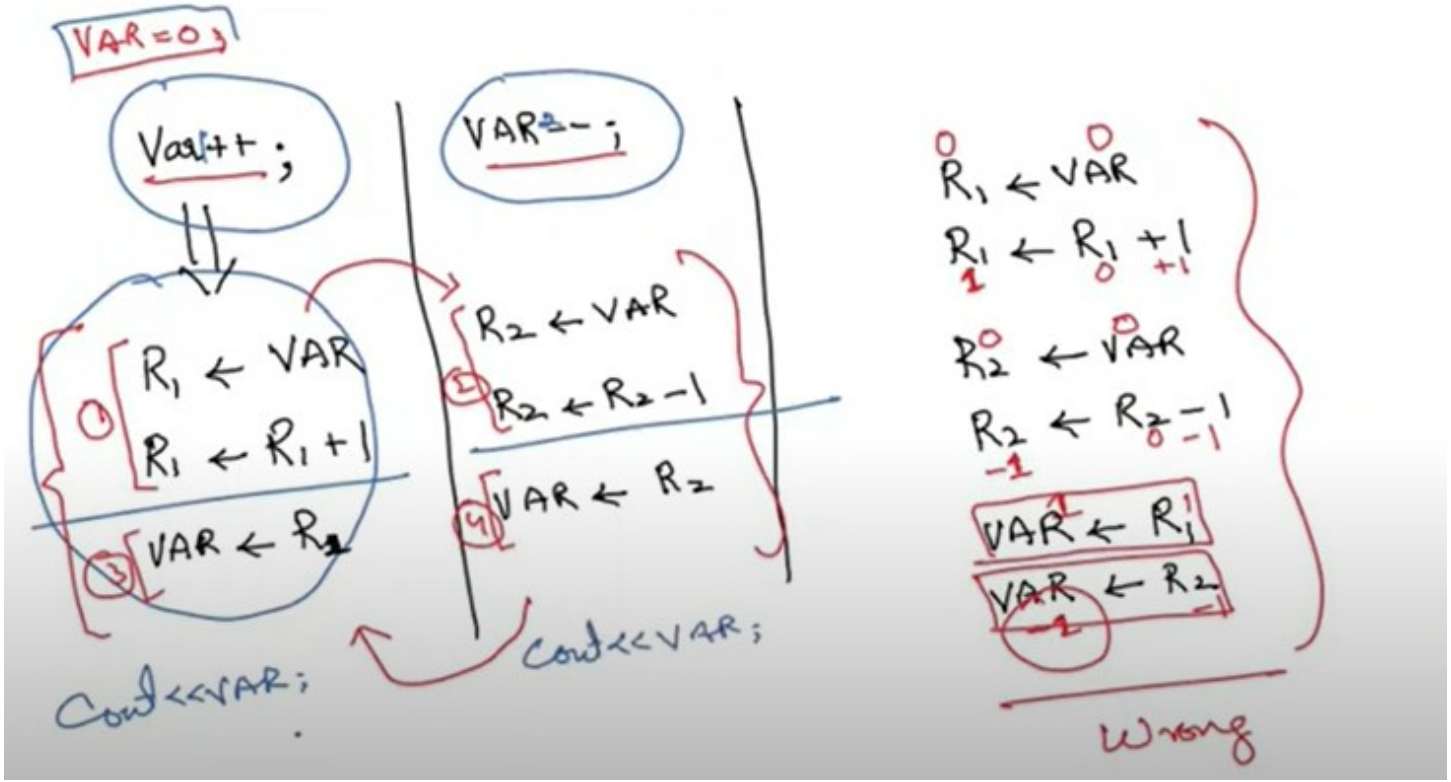
Cooperating Processes

- A process is cooperating if it can affect or be affected by the other processes executing in the system
- We provide an environment that allows process cooperation due to following reasons
- Information sharing
  - Many users or processes may be interested in the same piece of information
- Computation speedup
  - We breakup process into subtasks which run in parallel
- Modularity
  - We want to construct the system in modular fashion, dividing the system functions into separate threads
- Convenience
  - A user can do editing, printing and compiling in parallel

Critical Section

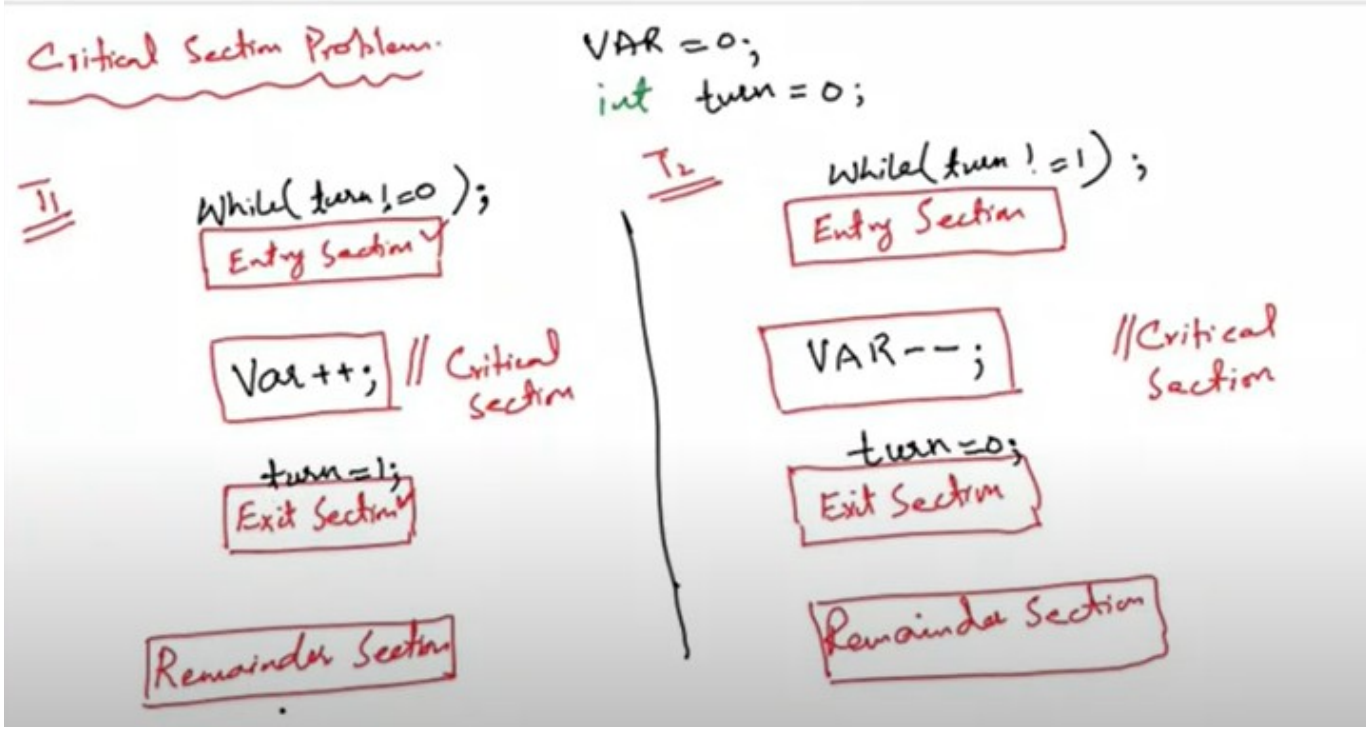
- Part of the program where shared memory is accessed is called critical section, or
- The area of a process which is sensitive to inter-process complications is called critical section or critical region
- Following are the four conditions to hold for a good solution
  - No two processes may be simultaneously in their critical sections
  - No assumptions may be made about speeds or number of CPUs
  - No process running outside its critical section may block other process
  - No process should have to wait forever to enter in its critical section

Critical Section



Critical Section Solution

- Both threads enter their critical section at the same time so the solution



Mutual Exclusion

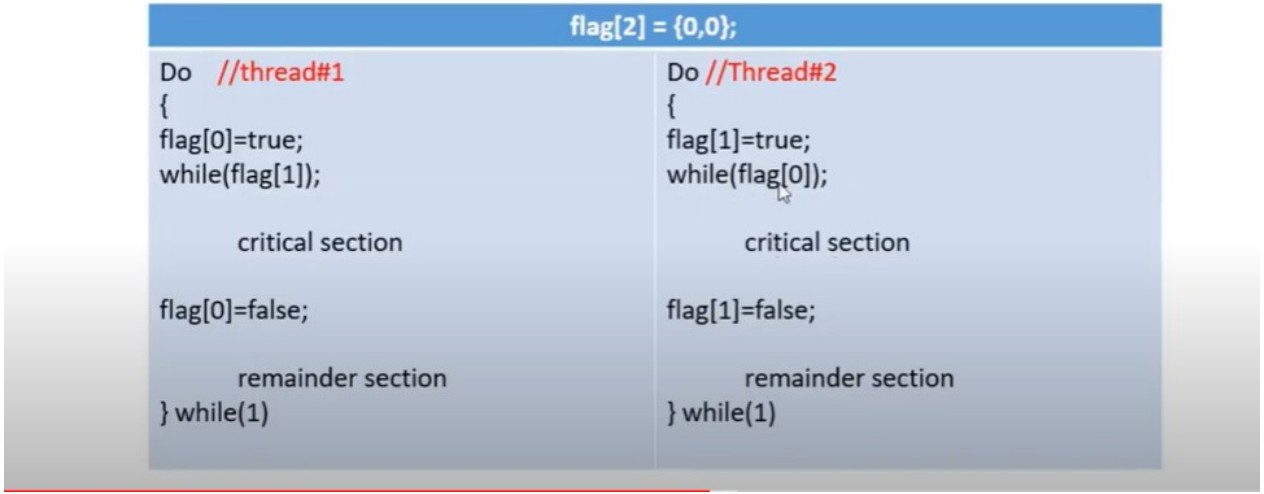
- A condition in which there is a set of processes, only one of which is able to access a given resource or perform a given function at any time
- Suppose two or more processes require access to a single non-sharable resource, such as printer
    - Each process will be sending data to the printer through critical section of each process
    - It is important that only one program at a time be allowed in its critical section
    - One cannot rely on operating system to enforce this restriction, therefore the processes have to be controlled

Solution to Critical Section Problem

- Mutual Exclusion
  - If process Pi is executing in its critical section, then no other processes can be executing in their critical sections
- Progress
  - Thread and/Process running in their remainder section should not participate in decision making
  - Decision making should not be indefinitely prolonged (timely manner)
  - If no process is executing in its critical section and there exist some processes that wish to enter their critical section, then the selection of the processes that will enter the critical section next cannot be postponed indefinitely
- Bounded Waiting
  - A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted
  - Assume that each process executes at a nonzero speed
  - No assumption concerning relative speed of the n processes

Solution to Critical Section Problem

Algorithm-1



flag[2] = {0,0};

Do //thread#1	Do //Thread#2
{	{
flag[0]=true;	flag[1]=true;
while(flag[1]);	while(flag[0]);
critical section	critical section
flag[0]=false;	flag[1]=false;
remainder section	remainder section
} while(1)	} while(1)

Solution to Critical Section Problem

Algorithm-2

turn = 0;

```
while(turn != 0);  //entry section

//critical section


turn = 1; //exit section

//remainder section<-==========
```

```
while(turn != 1);//entry section


//critical section


turn = 0;//exit section

//remainder section
```

Solution to Critical Section Problem

Algorithm-3

flag[2] = {0,0}; turn = 0;

```
Do  //flag[0] turn=0  //thread#
{
flag[0]=true;
turn=1;
while(flag[1] && turn==1){  }

      critical section

flag[0]=false;

      remainder section
} while(1)
```

```
Do   //flag[1] turn=1
{
flag[1]=true;
turn=0;
while(flag[0] && turn==0);

      critical section

flag[1]=false;

remainder section
} while(1)
```

Subtitles/closed captions (c)

Invalid solution critical section

1. Starvation (bounded waiting violation)
2. Deadlock (Progress violation)
3. Mutual exclusion violate

Peterson's Solution

```
do {
        flag[i] = true;
        turn = j;
        while (flag[j] && turn == j);

            critical section

        flag[i] = false;

            remainder section

} while (true);
```

Peterson's solution is restricted to two processes that alternate execution
between their critical sections and remainder sections.

```
int var = 0;  int flag[2] = {0,0}; int turn =0;
void* increment (void* ptr)
{
                        for (int i=0;i<1000000;i++)
                        {
                        flag[0] = 1;
                        turn=1;
                         while(flag[1]==1 && turn ==1);    //entry section
                            var++;          //critical section
                            flag[0]=0;       //exit section
                        }
}

void* decrement (void* ptr)
{
                        for (int i=0;i<1000000;i++)
                        {
                        flag[1]=1;
                        turn=0;
                         while(flag[0]==1 && turn ==1);       //entry section
      var--;            //critical section
                        flag[1]=0;          //exit section
                        }

}

int main()

{

pthread_t tid1,tid2;

pthread_create(&tid1,NULL,&increment,NULL);
pthread_create(&tid2,NULL,&decrement,NULL);

pthread_join(tid1,NULL);
pthread_join(tid2,NULL);
printf("%d\n",var);
return 0;

}
```

Hardware Solution

- Context switch is done by timer interrupt. If we disable the timer interrupt then time slice won't get expired, hence no synchronization issue.

- How to disable interrupts
  - CLI(Clear Interrupt Flag) instruction
  - SDI(Set Interrupt Flag) instruction
    - Flag register bit reserved for interrupt
  - They are privileged and cannot run in user mode

  - In Assembly language to do it "asm(cli)" if it works it will disable all of the keyboard and mouse

Hardware Solution

- CPU provides some instruction in its ISA that can execute atomically
- Atomically means its execution cannot be interrupted once its issued even on a multicore system.
- Such instruction once issued are committed.

Atomic Instruction

```
int compare_and_swap(int *value, int expected, int new_value) {
   int temp = *value;

   if (*value == expected)
      *value = new_value;

   return temp;
}
```

Use of atomic instruction

```
do {
   while (compare_and_swap(&lock, 0, 1) != 0)
      ; /* do nothing */

      /* critical section */

   lock = 0;

      /* remainder section */
} while (true);
```

Passing arguments

```
int sum1 = 0 , sum2 = 0;
void* add1 (void* ptr)
{
                int* num =(int*) ptr;
                for (int i=0;i<5;i++)
                                sum1 += num[i];
                return NULL;
}
void* add2 (void* ptr)
{
                int* num =(int*) ptr;
                for (int i=0;i<5;i++)
                                sum2 += num[i];
                return NULL;

}
int main()
{
                pthread_t tid1, tid2;
                int number[10] = {9,7,8,6,5,4,3,1,2,7};
                pthread_create(&tid1,NULL,&add1,(void*)number);
                pthread_create(&tid2,NULL,&add2,(void*)&number[5])
                pthread_join(tid1,NULL);
                pthread_join(tid2,NULL);
                int sum = sum1+sum2;
                printf("Sum is %d", sum);

return 0;

}
```

Microsoft says

Each process provides the resources needed to execute a program. A process has a virtual address space, executable code, open handles to system objects, a security context, a unique process identifier, environment variables, a priority class, minimum and maximum working set sizes, and at least one thread of execution.

- Each process is started with single thread, often called primary thread, it can create additional threads from any of its threads.
- Each process provides the resources needed to execute a program.
- All threads of a process share its virtual address space and system resources.
- In addition, each thread maintains exception handlers, a scheduling priority, thread local storage, a unique thread identifier, and a set of structures the system will use to save the thread context until it is scheduled.
- The *thread context* includes the thread's set of machine registers, the kernel stack, a thread environment block, and a user stack in the address space of the thread's process.
- Threads can also have their own security context, which can be used for impersonating clients.

https://docs.microsoft.com/en-us/windows/win32/procthread/about-processes-and-threads

Solaris threads

- Solaris 2 is based on Unix with support for threads at the kernel and user levels
- It introduce an intermediate level between kernel and user level threads which is called LWP(light weight process)
- Resource needs of threads;
  - **Kernel thread**
    - Small data structure and stack
    - Thread switching does not require changing memory access information
    - It is relatively fast
  - **LWP**
    - PCB with register data
    - Accounting and memory information
    - Switching between LWPs is relatively slow
  - **User level thread**
    - Only need stack and program counter
    - No kernel involvement means fast switching
    - Kernel only sees the LWPs that support user level threads

Solaris Threads VS POSIX Threads

- The Solaris threads API and the pthreads API are two solutions to the same problem: building parallelism into application software. Although each API is complete in itself, you can safely mix Solaris threads functions and pthread functions in the same program

- The two APIs do not match exactly, however. Solaris threads supports functions that are not found in pthreads, and pthreads includes functions that are not supported in the Solaris interface. For those functions that do match, the associated arguments might not, although the information content is effectively the same.

- By combining the two APIs, you can use features not found in one to enhance the other. Similarly, you can run applications using Solaris threads, exclusively, with applications using pthreads, exclusively, on the same system.



Thank you